

➤ Visit Frequency Distribution of the Customers

What it means:

The "visit frequency" of customers measures how often a customer visits the store within a specific time period. By plotting the frequency distribution, we can understand how many customers visit only once, twice, or multiple times. This helps analyze customer behavior and loyalty.

The **Visit Frequency Distribution of the Customers** analyzes how often customers visit or make purchases. Here's a simple breakdown of the graph and code for this part of the project:

Graph Explanation:

1. **X-axis (Visit_Frequency):** The number of times customers have made purchases or visits.
 - Example: Visit_Frequency = 1 means the customer visited or purchased once.
 2. **Y-axis (Count):** The number of customers who made visits corresponding to the X-axis value.
 - Example: Count = 349 means 349 customers visited exactly once.
 3. **Hover Information:**
 - **Visit_Frequency = 1:** The bar corresponds to customers with a single visit.
 - **Count = 349:** The height of the bar indicates 349 such customers.
 4. **Purpose of Analysis:**
 - Helps understand customer behavior.
 - Identifies frequent and infrequent visitors.
 - Businesses can target infrequent visitors with offers to increase visits.
-

elif selected_operation == 'Visit Frequency Distribution of the Customers':

- **What it does:**
 - Checks if the user selected "Visit Frequency Distribution of the Customers" from the dropdown menu.
 - If this is true, the code inside this block will execute.
-

```
Frequency =  
data.drop_duplicates(['Date','memberID']).groupby(by=['memberID'])['Date'].count().reset_index()
```

- **What it does:**
 - **Step 1:** `data.drop_duplicates(['Date', 'memberID'])`: Removes duplicate rows based on **Date** and **memberID** columns, ensuring each visit is counted once per day for each customer.

- **Step 2:** `.groupby(by=['memberID'])['Date'].count():`
Groups the data by **memberID** (unique customer identifier) and counts the number of unique dates each customer has visited (Visit Frequency).
 - **Step 3:** `.reset_index():`
Resets the groupby result into a DataFrame for further processing.
 - **Output:**
Creates a new DataFrame `Frequency` with two columns:
 - `memberID`: Unique identifier for each customer.
 - `Visit_Frequency`: Number of unique dates the customer has visited.
-

```
Frequency.columns = ['memberID', 'Visit_Frequency']
```

- **What it does**
 - Renames the columns of the `Frequency` DataFrame for better clarity:
 - `memberID`: Represents the customer ID.
 - `Visit_Frequency`: Represents the number of unique visits per customer.
-

```
Frequency.head()
```

What it does:

- Displays the first 5 rows of the `Frequency` DataFrame (for debugging or verification).
-

```
# Plot the histogram
```

```
fig = px.histogram(Frequency, x='Visit_Frequency', opacity=0.85, marginal='box')
```

What it does:

- Creates a histogram plot using Plotly Express (`px.histogram`):
 - **Frequency**: The DataFrame containing the data to be plotted.
 - **x='Visit_Frequency'**: Uses the `Visit_Frequency` column for the X-axis.
 - **opacity=0.85**: Adjusts the transparency of the bars (makes them slightly transparent).
 - **marginal='box'**: Adds a box plot above the histogram to show statistical distribution (e.g., median, quartiles).
-

```
fig.update_traces(marker=dict(line=dict(color='#000000', width=1)))
```

- **What it does:**

- Updates the layout of the histogram:
 - `title_text`: Sets the chart title to "Visit Frequency Distribution of the Customers."
 - `title_x=0.5`: Centers the title horizontally on the graph.
 - `title_font=dict(size=20)`: Sets the font size of the title to 20.
-

`st.plotly_chart(fig)`

What it does:

- Displays the histogram plot (`fig`) in the Streamlit app, allowing the user to interact with it.
-

➤ Monetary Distribution of the Customers

- **Monetary Distribution** refers to analyzing how much money customers spend. In this case, the monetary value is calculated based on the **number of items purchased** by customers and assuming each item has a fixed price.
- It shows the spending behavior of customers:
 - Some customers may spend more (frequent buyers or bulk purchasers).
 - Others may spend less (infrequent buyers or those purchasing fewer items).

This helps businesses understand customer spending habits, identify valuable customers, and tailor marketing strategies.

Graph Explanation

1. **X-Axis (Monetary)**: Represents the total amount of money spent by customers.
 - Calculated as `number of items purchased * item price (10 units)`.
 2. **Y-Axis (Count)**: Shows how many customers spent that particular amount.
 3. **Bar Heights**:
 - Each bar indicates the number of customers who fall into a specific monetary spending range.
 - For example, a bar at **60 units** with a count of **371** means **371 customers spent exactly 60 currency units**.
 4. **Box Plot (Above Histogram)**:
 - Summarizes the spread of monetary values:
 - **Median**: The middle value of spending.
 - **Quartiles**: How data is distributed (25%, 50%, 75%).
 - **Outliers**: Customers with spending far above the average.
-

Code Explanation (Line by Line)

Step 1: Check the Operation Selected

elif selected_operation == 'Monetary Distribution of the Customers':

- This checks if the user selected "Monetary Distribution of the Customers" from the dropdown menu in the Streamlit app.
- If true, the following block of code will execute.

Step 2: Group Data by Customers and Count Purchases

```
Monetary = data.groupby(by="memberID")['itemName'].count().reset_index()
```

- **data.groupby(by="memberID"):** Groups the dataset by **memberID** (customer ID).
- **['itemName'].count():** Counts the total number of items (itemName) purchased by each customer.
- **.reset_index():** Converts the grouped result into a DataFrame with default row indices.

Output:

Creates a DataFrame Monetary with:

- **memberID:** Unique identifier for each customer.
- **Count of Items Purchased:** Total number of items each customer bought.

Step 3: Rename Columns for Clarity

```
Monetary.columns = ['memberID', 'Monetary']
```

- Renames the columns in the Monetary DataFrame for better understanding:
 - memberID remains unchanged.
 - The count of items is renamed to Monetary.

Step 4: Display Data for Verification

```
Monetary.head()
```

- Shows the first 5 rows of the Monetary DataFrame for debugging or verification.

Step 5: Calculate Total Spending (Assuming Each Item Costs 10 Units)

```
Monetary['Monetary'] = Monetary['Monetary'] * 10
```

- Multiplies the count of items purchased (Monetary) by **10** (the assumed price per item) to calculate the **total monetary value** for each customer.

Step 6: Display Updated Data for Verification

Monetary.head()

- Displays the first 5 rows of the Monetary DataFrame after calculating the total monetary values.

Step 7: Plot the Histogram

```
fig = px.histogram(Monetary, x='Monetary', opacity=0.85, marginal='box',  
                  labels={'Monetary': 'Monetary'})
```

- Creates a histogram plot using Plotly Express:
 - **Monetary**: Data source.
 - **x='Monetary'**: Sets the X-axis to show the monetary values.
 - **opacity=0.85**: Makes the bars slightly transparent.
 - **marginal='box'**: Adds a box plot above the histogram to show statistical details.
 - **labels={'Monetary': 'Monetary'}**: Sets the label of the X-axis to "Monetary."

Step 8: Add Styling to the Histogram

```
fig.update_traces(marker=dict(line=dict(color='#000000', width=1)))
```

- Enhances the appearance of the histogram:
 - Adds a **black outline** around each bar for better visibility.
 - Sets the outline thickness to 1 unit.

Step 9: Update Chart Layout

```
fig.update_layout(title_text='Monetary Distribution of the Customers', title_x=0.5,  
                  title_font=dict(size=20))
```

- Configures the overall chart layout:
 - **title_text**: Sets the chart title.
 - **title_x=0.5**: Centers the title horizontally.
 - **title_font=dict(size=20)**: Sets the title font size to 20.

Step 10: Display the Chart in the Streamlit App

```
st.plotly_chart(fig)
```

- Displays the histogram (fig) in the Streamlit web app, allowing users to view and interact with the chart.

➤ RFM Scores/RFM Segments

RFM stands for **Recency**, **Frequency**, and **Monetary**—three key metrics used to analyze customer behavior.

1. **Recency (R):**
 - Measures how recently a customer made a purchase.
 - Customers who purchased recently are more likely to purchase again.
2. **Frequency (F):**
 - Measures how often a customer makes purchases.
 - Frequent buyers are considered loyal customers.
3. **Monetary (M):**
 - Measures how much money a customer spends.
 - Higher spending indicates valuable customers.

RFM Segmentation:

This involves classifying customers into meaningful categories (segments) based on their RFM scores. For example:

- **Champions:** Recent, frequent, and high spenders.
- **At Risk:** Previously valuable but haven't bought recently.
- **Loyal Customers:** Regular and frequent buyers.

Graph Explanation

The **treemap** represents the **distribution of RFM segments**:

1. **Each Box (Segment):** Represents a specific customer segment, e.g., "Potential Loyalists" or "At Risk."
 - The **size** of the box is proportional to the count of customers in that segment.
 - The **color** differentiates the segments.
2. **Hover Details:**
 - Displays the segment name, the count of customers, and the percentage of the total.

This visualization helps businesses understand their customer base distribution.

Code Explanation (Line by Line)

Step 1: Check Selected Operation

elif selected_operation == 'RFM Scores/RFM Segments':

- Executes this block if the user selects **RFM Scores/RFM Segments** from the dropdown menu.

Step 2: Calculate Recency

```
Recency = data.groupby(by='memberID')['Date'].max().reset_index()
Recency.columns = ['memberID', 'Recency']
```

- **data.groupby(by='memberID')['Date'].max():** Groups the data by memberID and finds the most recent purchase date for each customer.
- **Renaming Columns:** Changes to memberID and Recency.

Output:

A DataFrame showing how recently each customer made a purchase.

Step 3: Calculate Frequency

```
Frequency = data.drop_duplicates(['Date',
'memberID']).groupby(by=['memberID'])['Date'].count().reset_index()
Frequency.columns = ['memberID', 'Visit_Frequency']
```

- **Remove Duplicates:** Ensures each visit is counted only once per day.
- **Count Visits:** Groups by memberID and counts the unique visit dates.

Output:

A DataFrame showing how frequently each customer made purchases.

Step 4: Calculate Monetary

```
Monetary = data.groupby(by="memberID")['itemName'].count().reset_index()
Monetary.columns = ['memberID', 'Monetary']
Monetary['Monetary'] = Monetary['Monetary'] * 10
```

- **Group by memberID:** Calculates the total number of items purchased by each customer.
- **Monetary Value:** Multiplies the item count by 10 (assumed price per item).

Output:

A DataFrame showing how much each customer spent.

Step 5: Combine R, F, M DataFrames

```
RFM = pd.concat([Recency['memberID'], Recency['Recency'], Frequency['Visit_Frequency'],  
Monetary['Monetary']], axis=1)
```

- Combines the **Recency**, **Frequency**, and **Monetary** metrics into a single DataFrame.

Step 6: Calculate RFM Quartiles

```
RFM['Recency_quartile'] = pd.qcut(RFM['Recency'], 5, [5, 4, 3, 2, 1])  
RFM['Frequency_quartile'] = pd.qcut(RFM['Visit_Frequency'], 5, [1, 2, 3, 4, 5])
```

- **pd.qcut:** Divides Recency and Frequency into 5 equal segments (quartiles).
 - **Higher Recency Quartile (5):** Customer purchased recently.
 - **Higher Frequency Quartile (5):** Customer purchased more frequently.

Output:

Two additional columns:

- Recency_quartile
- Frequency_quartile

Step 7: Create RFM Score

```
RFM['RF_Score'] = RFM['Recency_quartile'].astype(str) +  
RFM['Frequency_quartile'].astype(str)
```

- Combines **Recency Quartile** and **Frequency Quartile** into a single RFM score (e.g., 55, 23).

Step 8: Map RFM Segments

```
segt_map = {  
    r'[1-2][1-2]': 'hibernating',  
    r'[1-2][3-4]': 'at risk',  
    r'[1-2]5': 'can\'t loose',  
    r'3[1-2]': 'about to sleep',  
    r'33': 'need attention',  
    r'[3-4][4-5]': 'loyal customers',  
    r'41': 'promising',  
    r'51': 'new customers',  
    r'[4-5][2-3]': 'potential loyalists',  
    r'5[4-5]': 'champions'  
}  
RFM['RF_Segment'] = RFM['RF_Score'].replace(segt_map, regex=True)
```


- Maps **RF_Score** values to meaningful **RFM Segments** using regex rules.

Example Mapping:

- 55 → champions
- 23 → at risk

Step 9: Count RFM Segments

```
x = RFM['RF_Segment'].value_counts()
```

- Counts the number of customers in each RFM segment.

Step 10: Plot Treemap

```
fig = px.treemap(x, path=[x.index], values=x)
fig.update_layout(title_text='Distribution of the RFM Segments', title_x=0.5,
title_font=dict(size=20))
fig.update_traces(textinfo="label+value+percent root")
st.plotly_chart(fig)
```

- Creates a **treemap** with:
 - **Path:** Segment names.
 - **Values:** Count of customers in each segment.
 - **Title:** "Distribution of the RFM Segments."
 - Adds **text details** showing labels, values, and percentages.